

# CUDA Bitonic Sort: Optimization and Performance Counter Analysis

## 1. Problem and approach

Bitonic sort executes  $p(p+1)/2$  compare-exchange stages for  $N = 2^p$ , each fully parallel; a stage at subsequence length  $k$  and stride  $s$  pairs element  $i$  with  $i+s$ . Inputs are padded to the next power of two with a sentinel above the maximum generated value, so padding sorts to the tail.

Everything below follows from one observation. At 100M, `pad_size = 227` gives 378 stages; one kernel per stage means 378 round trips through global memory, each reading and writing the whole array - ~400 GB of traffic for a 512 MB array, a ~120 ms floor at 3.35 TB/s against an 80 ms target. Per stage the arithmetic is two loads, a compare, two stores, so the kernel is bandwidth-bound and **every optimization reduces bytes moved or passes taken, not arithmetic.**

## 2. Optimization 1: shared-memory tiling

A block loads `TILE` contiguous, `TILE`-aligned elements into shared memory. Both members of every pair at stride  $s$  fall inside that window exactly when  $2s \leq \text{TILE}$ , so all such stages need no inter-block communication. This splits the work into two kernels:

- **Kernel A** (`bitonic_merge_small_k`) handles every  $k \leq \text{TILE}$ . Each such  $k$  has all strides  $\leq k/2 \leq \text{TILE}/2$ , so the entire triangle of stages runs in **one launch**: load tile, loop  $k$  and stride with `__syncthreads` between stages, write once.
- **Kernel B** (`bitonic_merge_large_k`) handles  $k > \text{TILE}$ . Strides  $\geq \text{TILE}$  must go through global memory one stage per launch; once the stride falls to  $\text{TILE}/2$  the remaining cascade for that  $k$  completes in one shared-memory launch.

At `TILE = 16384` this covers all 378 stages in 105 launches. Kernel B takes  $k$  as an argument because direction depends on global, not tile-local, position; since  $k \geq 2 \cdot \text{TILE}$  there, a whole tile shares one direction, so `dir` is computed **once per launch** rather than per pair.

## 3. Optimization 2: data type

`main.cu` fills with `rand() % 1000`, so values never exceed 999 and `short` (max 32,767) is a valid `DTYPE`. This halves H2D, D2H and kernel traffic at once - an exact 2.0x on both transfers - and halves shared memory per tile, so `TILE` doubles to 16384 in the same 48 KB budget, removing 14 more launches. One effect was unplanned: bank conflicts collapsed from 96.0 M to 0.33 M, since two 16-bit elements share a 32-bit bank word and the low-stride pattern now resolves within one word.

## 4. Optimization 3: transfers

D2H initially cost 130 ms. Switching the output buffer to `cudaHostAlloc` barely helped: pinning 400 MB itself costs ~83 ms, merely replacing the page-fault cost. The fix is **overlap** - pinning is host-side work that does not block the GPU, so issuing the sort kernels first (asynchronous) and allocating the pinned buffer while the GPU is busy hides it entirely behind kernel time, taking D2H from 130 ms to 3.6 ms.

H2D stays at ~9.6 GB/s because `arrCpu` is `malloc'd` in `main.cu`, so every transfer is staged through a driver-internal pinned buffer. `cudaHostRegister` page-locks it in place for ~7.6 ms per 200 MB, ~5x cheaper per byte than allocating pinned memory, letting DMA read `arrCpu` directly.

## 5. Optimization 4: address arithmetic

`ncu` showed kernels A and B at 75% compute but only 25% memory throughput, with `math_pipe_throttle` at ~30% of warp cycles - compute-bound, not memory-bound. The cause is  $i =$

$(t/stride)*2*stride + (t\%stride)$  and  $dir = ((base+i)/k) \% 2$ : with no hardware integer divide, a variable divisor compiles to a MUFU.RCP + I2F/F2I + multiply-high sequence, leaving shared loads/stores just 86 of 615 SASS instructions. Both divisors are powers of two, so  $2t - (t \& (stride-1))$  and  $(g \gg \log2k) \& 1$  are equivalent, cutting 23% of `small_k`'s instructions and 16% of kernel time.

That rewrite pushed registers 32 to 40 and halved occupancy: at 1024 threads, 32 registers exactly fills the 65,536-register file for two blocks, so 40 admits one. `__launch_bounds__(1024, 2)` restores 32 with zero spilling.

## 6. Optimization 5: vectorized global merge

The global stage kernel was still latency-bound (long-scoreboard 31%). Loading `int4` (8 shorts per thread) raised sectors per request from 2.00 to 16.00 - each warp instruction now moves 512 B instead of 64 B for the same total bytes. Memory throughput on that kernel rose 30% to 66% and total kernel time fell 44%, from 7.16 ms to 3.98 ms at 10M.

## 7. Results

All runs on H100, `grade.py`, 100M elements.

| Configuration                                     | H2D (ms) | Kernel (ms) | D2H (ms) | meps | Occupancy | Score        |
|---------------------------------------------------|----------|-------------|----------|------|-----------|--------------|
| Naive, one kernel per stage, <code>int</code>     | 42.3     | 123.9       | 130.3    | 337  | 41.8%     | 13.16        |
| + shared-memory tiling                            | 40.5     | 73.5        | 129.4    | 409  | 58.0%     | 16.70        |
| + <code>short</code> , TILE 16384, pinned overlap | 21.1     | 61.2        | 3.6      | 1163 | 69.4%     | <b>21.00</b> |
| + <code>int4</code> vectorized global merge       | 20.8     | 62.2        | 3.6      | 1146 | 67.5%     | 21.00        |

Net: 337 to 1163 meps, a 3.4x improvement, with D2H down 36x and the kernel down 2.0x.

## 8. Performance counters

`ncu --set full` with explicit metrics, 10M elements, exported from the `.ncu-rep` (67 launches).

| Kernel                              | Launches  | Duration (ms) | % time        | SM thr. | Mem thr. | Achieved occ. | Theoretical occ. |
|-------------------------------------|-----------|---------------|---------------|---------|----------|---------------|------------------|
| <code>bitonic_merge_small_k</code>  | 1         | 1.621         | 40.7%         | 78.20%  | 33.06%   | 93.78%        | 100.0%           |
| <code>bitonic_merge_vec_int4</code> | 55        | 1.123         | 28.2%         | 31.75%  | 66.35%   | 73.31%        | 100.0%           |
| <code>bitonic_merge_large_k</code>  | 10        | 1.205         | 30.3%         | 78.91%  | 68.81%   | 93.64%        | 100.0%           |
| <code>fill_padding</code>           | 1         | 0.034         | 0.9%          | 23.70%  | 9.57%    | 9.50%         | 100.0%           |
| <b>Total</b>                        | <b>67</b> | <b>3.984</b>  | <b>100.0%</b> | —       | —        | —             | —                |

Required counters, summed over all launches:

| Kernel                              | Global LD sectors (M) | Global ST sectors (M) | Sectors/request | Local LD/ST | Divergent branches | Achieved occ. |
|-------------------------------------|-----------------------|-----------------------|-----------------|-------------|--------------------|---------------|
| <code>bitonic_merge_small_k</code>  | 1.0                   | 1.0                   | 2.00            | 0           | 0                  | 93.78%        |
| <code>bitonic_merge_vec_int4</code> | 57.7                  | 57.4                  | 16.00           | 0           | 0                  | 73.31%        |
| <code>bitonic_merge_large_k</code>  | 10.5                  | 10.5                  | 2.00            | 0           | 0                  | 93.64%        |
| <code>fill_padding</code>           | 0.0                   | 0.4                   | —               | 0           | 0                  | 9.50%         |
| <b>Total</b>                        | <b>69.2</b>           | <b>69.4</b>           | <b>7.38</b>     | <b>0</b>    | <b>0</b>           | —             |

Local memory traffic is zero, confirming no register spilling, and divergent branches are zero (`thread_inst_executed_per_inst_executed = 32.00`) because the compare-exchange compiles to predicated selects. Stall data separates the two kernel classes: the shared kernels are limited by `not_selected` (~37%) and `math_pipe_throttle` (~20-28%), while the vectorized global kernel is `long_scoreboard`-bound at 67% with 0.94 eligible warps per scheduler - genuinely DRAM-bound, the intended end state.

## 9. Observations and next steps

Memory Throughput reads 44%, short of the 75% target, but this is the optimization working rather than a regression: the shared-memory kernels deliberately avoid DRAM, so raising that number would mean undoing the tiling.

Two further steps were identified but not implemented. Register blocking in the global kernel - each thread resolving  $b$  consecutive stride levels before writing back - would cut 55 global launches to ~21 at  $b = 4$  and traffic from 48 GB to 19 GB. Separately, `fill_padding` is 0.9% of runtime yet its 9.50% occupancy drags the unweighted per-kernel average behind the occupancy score; a `cudaMemset` does not help, since that still launches an internal kernel the profiler counts.

## References

1. CUDA Runtime API, Memory Management. [https://docs.nvidia.com/cuda/cuda-runtime-api/group\\_\\_CUDART\\_\\_MEMORY.html](https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__MEMORY.html)
2. How to Optimize Data Transfers in CUDA C/C++. <https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/>
3. Boosting Application Performance with GPU Memory Access Tuning. <https://developer.nvidia.com/blog/boosting-application-performance-with-gpu-memory-access-tuning/>
4. Nsight Compute Profiling Guide. <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html>